Beamline: Multi-Agent Design, Verification, and Fabrication of Tabletop Optical Systems from Natural Language and Published Figures

Beamline Team

Hackathon submission, 25 October 2026

Abstract

Laying out a free-space laser-optics bench for atomic, molecular and optical (AMO) physics is slow, manual and error-prone: beams must be routed, split, frequency-shifted and coupled into fibers while every stray reflection is terminated for laser safety. We present *Beamline*, a system built in a single hackathon day in which teams of large-language-model (LLM) agents propose optical layouts—from a natural-language goal, from a figure in a published paper, or from a goal so large that it must be decomposed into subsystems—and a deterministic physics core disposes of them. The core is a planar Gaussian-beam and Jones-calculus tracer (including acousto-optic diffraction orders, fiber mode overlap, and self-consistent nonlinear elements) coupled to a rule-based verifier that scores designs and emits machine-readable errors that drive repair. Verified designs are tuned by a deterministic alignment search and exported as a fabrication kit (bill of materials, keyed and tiled 3D-printable baseplates, a PyOpticL script) and a typeset build document. On the day we measured: grounding the model in real library code reduced API hallucination in generated layout code from 95–98% to 0%; figure-to-design reproduction of a saturated-absorption spectroscopy figure passed verification in 3/3 concurrent runs (41 s to 55 s); on an eight-figure stress test of open-access papers 4/8 reproductions passed verification and 3/8 were physically faithful while the goal pipeline passed 4/4; a 121-element trapped-ion apparatus traces and verifies in 6–10 ms; and in two live end-to-end runs, a paper search to a finished build document took 1:23–1:40 against a 3:00 budget. We report the failures in detail and discuss what they imply for multi-agent systems in complex optical design: decomposition that mirrors how laboratories modularize, parallel exploration against objective verifiers, consensus for perception, and deterministic tools as the ground truth that LLMs must answer to.

1 Introduction

An AMO experiment such as a trapped-ion quantum processor [1] or an optical tweezer [2, 3] sits on top of one or more optical tables carrying tens to hundreds of mirrors, lenses, waveplates, polarizing beam splitters (PBS), acousto-optic modulators (AOMs) and fiber couplers. Standard building blocks recur—double-pass AOMs [4], Doppler-free saturated absorption spectroscopy (SAS) [5], Pound–Drever–Hall (PDH) locks [6]—but every laboratory re-derives the layout by hand. Mistakes are found late, on the table: a beam clipped on a mount, a reflected order that leaves the table carrying milliwatts, a waveplate set so that a PBS sends the power to the wrong port, a fiber coupler that is mode-mismatched. Code-to-CAD tools such as PyOpticL [7] have shown that describing an optical layout as a program that places optics *along simulated beam paths* makes designs reproducible, modular and directly fabricable as 3D-printed, "paint by numbers" baseplates. Writing that program, however, still requires an expert.

LLMs can now read figures, write code and follow long technical instructions, and agentic systems have begun to plan and run experiments in chemistry [8, 9] and to write papers [10]. Free-space optics is an attractive but unforgiving target: a design is either physically consistent or it is not, and plausible-looking prose or code is cheap. Our working principle is therefore *LLMs propose, deterministic physics disposes*: every model output is a candidate that must survive a tracer and a verifier, whose diagnostics are fed back for repair.

Contributions. In one hackathon day (Section 3.8) we built and evaluated:

1. A compact JSON design intermediate representation (IR) for planar tabletop optics with beam-relative placement and deterministic branch naming (Section 3.1).
2. A fast planar Gaussian/Jones tracer with AOM orders, fiber mode-overlap coupling, clipping and self-consistent nonlinear elements (semiconductor optical amplifier, sum-frequency generation, saturable absorber), and a verifier that produces a score and

actionable errors (Sections 3.2–3.3).

3. Three multi-agent pipelines on top of Gemini (`gemini-4-argon`) [11]: goal→design with best-of-N candidates and verifier-in-the-loop repair; figure→design with three parallel vision agents, consensus and mm scaling; and a hierarchical architect with parallel module agents and a deterministic laser power budget (Section 4).
4. A deterministic stray-beam autofix, an alignment tuner, and a fabrication path to a bill of materials, watertight keyed baseplate STL tiles, a PyOpticL export and a typeset build document (Sections 3.4–3.6).
5. An honest evaluation: a grounding study, reliability runs, an eight-figure benchmark including failures, scaling, tuning recovery, and optical-logic truth tables (Section 5), followed by a discussion of the potential and pitfalls of multi-agent systems for optical design (Section 6).

2 Related work

Code-to-CAD optics and simulation. PyOpticL [7] places optics along simulated, branching beam paths in Python and generates modular CAD for printed or machined baseplates with pre-aligned mounts; Beamline adopts its beam-relative placement idea, targets it through an exporter, and reproduces a figure from its paper. Interferometer simulators such as Finesse [12] model coherent fields in detail; our tracer is deliberately simpler (incoherent, planar) and optimized for millisecond feedback to agents. Our schematic rendering vocabulary follows the ComponentLibrary convention common in optics figures [13].

Computer-designed experiments. MELVIN [14], active-learning agents [15] and PyTheus [16] search over representations of quantum-optics experiments, raising the question of what scientific understanding such systems provide [17]; recently, the LLM agent AI-Mandel was given access to such tools [18]. These systems target abstract quantum-state generation; Beamline targets the physical layout of classical beam-delivery optics that every AMO laboratory builds.

LLM agents for optical design. Uddin et al.'s AI-driven robotics platform for optics [19] uses generative AI to turn goals into free-space optical configurations that a robotic arm assembles, aligns and measures; Anubuddhi [20] is a multi-agent system for designing and simulating quantum-optics experiments; OPTIAGENT [21] trains an LLM for lens design with reinforcement learning (DrGRPO) and a physics-driven agentic framework. Earlier, inverse design reshaped nanophotonics [22], a contrast to the discrete, component-level layout problem we address. Beamline differs in combining figure-based reproduction with a verifier-scored repair loop, hierarchical decomposition with physical power accounting, and a fabrication output.

LLM agents for science and agent patterns. AI is increasingly part of scientific discovery [23], and LLM-based autonomous agents are a fast-growing field [24]. Coscientist [8], ChemCrow [9] and the AI Scientist [10] couple LLMs with domain tools. Our loops instantiate familiar patterns: reasoning interleaved with tool calls (ReAct [25]), verbal feedback from failures (Reflexion [26], Self-Refine [27]), and role-specialized multi-agent conversations (AutoGen [28], MetaGPT [29]). The distinguishing feature here is that the critic is not an LLM but a physics engine.

Physics. Beam propagation uses the complex beam parameter q with ABCD matrices [30–32]; polarization uses Jones calculus [33]. Domain blocks follow standard references: AOM double pass [4], SAS [5], PDH [6], tweezers [2, 3], trapped ions [1]; nonlinear elements follow textbook SOA gain saturation [34], quasi-phase-matched $\chi^{(2)}$ conversion [35] and a single-PPLN half adder proposed (and simulated) by Wang et al. [36].

3 System

3.1 Design IR

A `Design` (`backend/optics/ir.py`) contains a baseplate, sources (position, direction, wavelength, power, waist, Jones polarization, enable switch), an ordered element list, optional targets (e.g. "≥5 mW into fiber F1", polarization fidelity), optional modules (rectangular regions) and an optional `LogicSpec`. Units are mm, degrees, nm, mW and MHz; the table frame has its origin at the baseplate's lower-left corner. Elements are placed either absolutely (`at`) or, PyOpticL-style, *on a beam*: `beam` names a branch and `distance_mm` the distance along it. Branch names are deterministic: a source starts branch `L1`; splitters append `.t`/`.r`; AOMs append `.0` and `.+1`/`.-1`; other elements keep the name (e.g. `L1.r.+1`). Mirrors may be specified by the desired `out_direction_deg`, which the tracer converts into a surface normal. This makes designs short, mostly free of trigonometry, and robust to moving an upstream optic—properties that matter when an LLM writes them.

3.2 Planar Gaussian + Jones tracer

`trace(design)` (`optics/trace.py`) runs two passes. A *placement pass* traces every source depth-first and places the first not-yet-placed element on the current branch at its distance, unless an already-positioned element is physically intersected first (repeated up to six rounds so combiners on later sources' branches are seen). A *physics pass* freezes geometry and traces by ray–segment

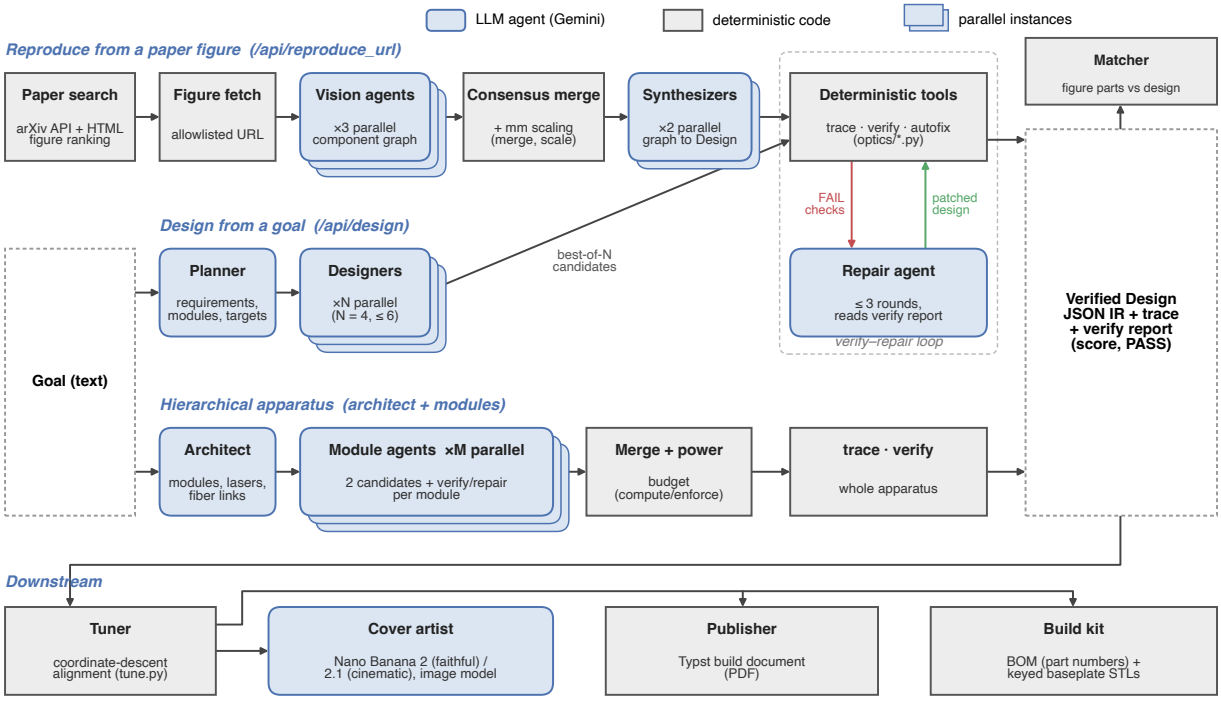


Figure 1: Beamline architecture. Gemini agents (blue) propose designs; deterministic code (grey) evaluates, repairs and fabricates them; stacked boxes are parallel instances. Three entry paths (figure reproduction with 3 vision agents, consensus merge and 2 synthesizers; goal→design with N=4 designers; hierarchical architect with M module agents under a shared power budget) feed a verify–repair loop, then the tuner, cover artist, Typst publisher and build-kit generator. Every agent state change, model call, streamed token batch, tool run and intermediate design is emitted as a server-sent event and rendered in a live console.

intersection, producing segments, hits, powers and coupling numbers; branches below 10^{-6} mW are dropped. Each element is a line segment (splitter cubes use their diagonal, so they can be hit from all faces). Gaussian beams carry q through thin lenses [30, 31]; polarization is a Jones vector in the beam frame with sign flips on reflection and mirrored fast axes for counter-propagating waveplates [33]. Clipping on a circular aperture of radius r for beam radius w and offset h uses $(1 - e^{-2r^2/w^2})\,T_x(h)/T_x(0)$ with an erf-based offset correction.

AOMs and fibers. An AOM diffracts order ± 1 by $\theta = \lambda f / v$ with efficiency η_{AOM}, leaving $(1 - \eta_{\mathrm{AOM}})$ in the zero order, so double-pass geometries [4] trace naturally. A fiber coupler focuses the beam with its lens ($f = 11$ mm default) onto a fiber of mode radius w_f and computes

$$\eta = \eta_{\mathrm{ovl}}\, e^{-(f\alpha/w_f)^2}\, e^{-(\pi w_f h/\lambda f)^2}\, T_{\mathrm{clip}}, \tag{1}$$

where η_{ovl} is the Gaussian size-and-curvature overlap, α the tilt and h the offset at the lens.

Nonlinear elements. For all-optical logic the IR includes an SOA, a $\chi^{(2)}$ crystal and a saturable absorber. If any are present the physics pass iterates to a fixed point: trace, sum incident power P_{tot} per element, update state, retrace; convergence at 10^{-4} relative change within 12 iterations (damped by 50% from iteration 5). The SOA applies a shared gain $G = G_0/(1 + P_{\mathrm{tot}}/P_{\mathrm{sat}})$, giving cross-gain modulation [34]; the absorber transmits $T = T_0 + (1 - T_0)P_{\mathrm{tot}}/(P_{\mathrm{tot}} + P_{\mathrm{sat}})$. The SFG crystal combines co-propagating, overlapping beams into $1/\lambda_{\mathrm{out}} = 1/\lambda_1 + 1/\lambda_2$ with a photon-number-limited power

$$P_{\mathrm{out}} = Q_{\min} \tanh^2 \sqrt{\eta P_1 P_2 \cdot 10^{-3}/Q_{\min}}, \quad Q_i = P_i \lambda_i / \lambda_{\mathrm{out}}, \tag{2}$$

depleting each input by one photon per generated photon. Phase matching is assumed perfect (quasi-phase-matched [35] for the configured process only).

3.3 Verifier and score

`verify(design, trace)` (`optics/verify.py`) emits checks with severities: unplaced elements; clipping (warning $> 1\%$, error $> 10\%$); stray beams (error if an escaping segment carries > 0.5 mW, "laser safety: unterminated beam"); elements off the table; footprint collisions; beams passing through another element's mount without interacting; unmet targets; trace depth caps; dark elements; and nonlinear non-convergence. Footprints are per-type rectangles (e.g. mirror 25×25 mm, AOM

40×25 mm). The score is

$$S = \mathrm{clip}_{[0,100]}\Big(95 + 5\,\tfrac{P_{\mathrm{delivered}}}{P_{\mathrm{in}}} - 25\,n_{\mathrm{err}} - 5\,n_{\mathrm{warn}}\Big), \quad (3)$$

with delivered power counting fiber-coupled and photodiode-absorbed light (beam dumps excluded). A design *passes* when it has no errors. Every message names elements and branches, so it can be pasted verbatim into a repair prompt.

3.4 Deterministic autofix

Stray beams were the most common error in early runs (Section 5.3). `optics/autofix.py` therefore terminates every escaping branch carrying non-trivial power by trying beam-dump placements (near/far, full block or knife edge) just inside the table edge and keeping the first that terminates the branch *without adding any error*—so a dump never clips a wanted beam running nearby. It runs before every LLM repair round and frees the model for real topology problems.

3.5 Alignment tuner

`optics/tune.py` performs coordinate search over physical knobs in beam order: waveplate fast axes, lens and coupler distances, coupler focal length, mirror angles ($\pm 1.5°$), and split ratios; coarse grid then refinement; a step is kept only if it strictly improves the objective. The objective rewards target satisfaction, delivered fraction, fiber coupling and score, and penalizes structural errors so the search never "dims the light" to silence stray-beam errors. A companion `detune()` produces realistically misaligned copies (waveplates off by 8–25°, lenses by 8–20%, mirrors by 0.2–0.8°) for demonstration and testing.

3.6 Build kit and build document

`optics/bom.py` produces a bill of materials (an element's own part number wins, otherwise a default optic+mount kit with list-price *estimates*) and a printable baseplate built with manifold3d [37] booleans: 1/4-20 table holes on a 50.8 mm grid, a counterbored screw hole and two 2 mm dowel-pin holes per element oriented to its optical axis (so mounts are keyed and pre-aligned, as in PyOpticL), and a 1 mm groove along every beam segment. Plates larger than 250 mm are tiled with half-lap joints. The kit (zip) contains `BOM.csv`, STL tiles, a PyOpticL/FreeCAD script and an assembly README. `optics/build_doc.py` typesets a PDF with Typst [38] (cover, plan-view SVG, verification, BOM, assembly steps in beam order, alignment procedure from the tuner, laser safety, credits); compilation typically takes ∼0.3 s. All agent-written text reaches Typst as data, never as markup.

Table 1: System components (all in `backend/` unless noted).

Component	Role
`optics/ir.py`	Pydantic design IR, trace/report schemas
`optics/trace.py`	Planar Gaussian+Jones tracer, AOM, fiber, nonlinear fixed point
`optics/verify.py`	Checks, severities, score
`optics/autofix.py`	Validated stray-beam termination
`optics/tune.py`	Coordinate-search alignment, detune
`optics/merge.py`	Namespaced merge of module designs
`optics/bom.py`	BOM, keyed tiled baseplate STL
`optics/build_doc.py`	Typst build document
`optics/pyopticl_export.py`	Export to PyOpticL source
`agents/design.py`	Goal→design, best-of-N + repair
`agents/reproduce.py`	Figure→design, 3 vision agents
`agents/hierarchical.py`	Architect + module agents + budget
`agents/papers.py`	arXiv search, figure scoring
`agents/events.py`, `llm.py`	Console events, streamed Gemini calls
`agents/cover.py`	Nano Banana cover renders
`web/`	three.js viewer, console, tuning tour

3.7 Web application

A Vite/React/three.js [39] app renders the bench in 3D with live re-tracing on edit (`/api/trace`), a verify panel, an inspector, module regions, a logic I/O strip, and an agent console that shows each agent's state, streamed tokens and thought summaries, tool runs and an iteration timeline with deterministic design diffs. In figure mode the source figure lies above the table and recognized parts "fly in" from their detected bounding boxes; after a passing design, the app auto-detunes and runs a tuning tour with a live power meter, renders cover art with Nano Banana image models, and produces the build PDF.

3.8 Provenance and hackathon scope

Everything in Table 2 labelled "built today" was written on 25 October 2026 during the hackathon. The git history of the `beamline` repository starts at 11:23 PDT that day (commit `4817b5d`, "Contract: design IR, trace/report schemas, API spec") and comprised 48 commits at the time of writing. The code was written by an AI coding assistant (Claude Code) and multiple sub-agents running during the event under the team's direction; the team set goals, reviewed results, and made design decisions. This paper, its figures and its bibliography were likewise drafted by AI agents during the event under the team's direction.

Table 2: What was built during the hackathon versus what pre-existed.

Category	Items
Built today, from scratch (`beamline` repo, 48 commits from 11:23 PDT)	Design IR; tracer and verifier incl. nonlinear models; autofix; tuner; merge and hierarchical pipeline; all agent pipelines (design, reproduce, hierarchical, paper search, console events, token streaming); PyOpticL exporter; BOM/STL/build-document generation; cover-render integration; web app (three.js viewer, console, fly-in, tuning tour); examples and benchmark set; the evaluation in this paper.
Built today, outside the app	Gemini latency benchmark tool (`gembench.py`); `gemini-4-argon` capability probe (`experiments/argon_probe`).
Pre-existing, used	PyOpticL [7] (design concept and export target; cloned for reference, not vendored); Gemini models (`gemini-4-argon`, Nano Banana 2/2.1, TTS) via API; open-source libraries (three.js, React, @react-three/fiber, drei, FastAPI, Pydantic, NumPy, google-genai, httpx, manifold3d, trimesh, Pillow, Typst, ffmpeg); published figures used as inputs (CC BY/CC0, and the arXiv-licensed Gieseler et al. figure [3], fetched live and not redistributed).

4 Multi-agent pipelines

All agents call `gemini-4-argon` through a shared wrapper (`agents/llm.py`) that always sets the thinking level explicitly, allows up to 32,768 output tokens, requests schema-constrained JSON validated into the Pydantic IR (with one corrective retry that feeds back the validation error), and streams both output and thought summaries. The guiding rule is that no model output is trusted: every design goes through `trace`+`verify`, and every arithmetic quantity that matters (scales, power budgets, region packing) is computed deterministically.

4.1 Goal → design

`agents/design.py`: a planner drafts an approach; $N = 4$ designer agents generate candidates in parallel (a straggler grace of 15 s after the first lands, 75 s hard cap); each is traced and verified; the best enters a repair loop (`agents/common.py`) that first applies autofix and then gives the model the verify report plus a compact trace digest (branch list, powers, hits). Error rounds always run while time remains; warnings-only "polish" rounds start only with enough budget left. Every candidate and repair round is emitted as an `iteration` event with a deterministic diff against its parent.

4.2 Figure → design

`agents/reproduce.py` reproduces a setup from an image: three vision agents run in parallel with different emphases (exhaustive parts list, beam connectivity, labels and values), each returning parts with bounding boxes in normalized 0–1000 coordinates and beam links. A merge step takes a majority vote on the order-independent parts signature; once two passes agree the third gets only 4 s more. A deterministic scaling step maps pixels to millimetres using known mount sizes (1" ≈ 38 mm) and stretches the layout so optic centres are ≥ 40 mm apart, because our probe showed drawings read 1.5–3× too short. Two synthesizer agents convert the graph into a traceable IR in parallel; the best is repaired under a hard deadline (in the current code the final design is always emitted within 115 s of the request, with at most one repair round after 75 s; some runs reported below predate this setting). A deterministic matcher pairs figure components with design elements for the UI. Figures can be dropped in or found by `agents/papers.py`, which queries the arXiv API, scrapes figures and captions from arXiv HTML, ranks them by an "optical setup" caption score and reports the license; downloads are restricted to an allowlist of hosts.

4.3 Hierarchical design

`agents/hierarchical.py` handles apparatus with 50–200 optics. An architect agent (high and medium thinking raced; medium wins after an 8 s grace) splits the goal into fiber-in/fiber-out modules with sizes, purposes and links, and lists each physical laser once. Regions are shelf-packed deterministically. A deterministic budget then derives each module's input power: root modules receive the power of lasers they host, outgoing feeds are capped at 90% of a module's input, fiber feeds are multiplied by 0.7. Module agents design all modules in parallel in local frames (two candidates each, then repair); designs are rescaled to their budget and merged with namespaced ids (`optics/merge.py`), then the whole is traced and verified, with `power_budget` checks added to the report.

4.4 Observability

Each pipeline emits a fine-grained event stream (`CONTRACT.md`): agent state changes, per-call token counts and latencies, streamed thought summaries, raw token batches every ∼120 ms, deterministic tool runs and intermediate designs with diffs. The console shows parallel agents as colored token lanes. Beyond the demo value, this stream was the primary debugging instrument: every failure analysed in Section 5.3 was diagnosed from recorded streams.

5 Results

All numbers below were measured on 25 October 2026 on a laptop (macOS, Apple silicon) against the live Gemini API; raw event streams are in the repository.

Table 3: Eight-figure stress test (25 Oct 2026, `demo/figures/RESULTS.md`). Entries: time / score / pass / number of elements. "–" = not attempted.

Figure (license)	Diff.	Reproduce	Design	Notable failure
Raman backscatter (CC BY 4.0)	easy	46 s / 96 / pass / 12	–	Sample modelled as a mirror; filter passes laser line to CCD (physically wrong pass)
Pump–probe delay (CC0)	easy	55 s / 100 / pass / 8	–	First synth left a 50 mW stray; repair 1 broke a mirror; fixed in round 2
PDH thin-disk lock (CC BY 4.0)	medium	41 s / 95 / pass / 9	52 s / 99.6 / pass / 14	No cavity element; resonator not modelled
Sr^+ ion-trap lasers (CC BY 4.0)	medium	104 s / 0 / fail / 27	68 s / 74.7 / pass / 21	9 unterminated AOM/sampler ports; budget exhausted before repair
MOT + LCVR laser system (CC BY 4.0)	medium	90 s / 0 / fail / 22	aborted (infra)	Dump on diffracted order, then optics placed on it; 14 cascaded branch errors
Dual-trap tweezers, arXiv:2411.16256 (CC BY 4.0)	hard	80 s / 96 / pass / 22	84 s / 99.5 / pass / 58	QPDs on forward beam; focal lengths and 3 W from notes ignored; design used 4 separate sources
HOM tomography (CC BY 4.0, 3D)	hard	119 s / 18 / fail / 41	–	Vision 75.6 s; LLM merge failed; 16% clipping; no time to repair
Yb^+ clock laser (CC BY 4.0, dense)	hard	error ×2 at 60 s	–	60 s vision deadline cancelled all passes
MALDI ion source (goal only)	medium	–	50 s / 99.2 / pass / 32	Target is a fiber coupler, not a focused spot

5.1 Grounding study

Before building the agents we probed `gemini-4-argon` on 14 requests (`experiments/argon_probe/FINDINGS.md`; Fig. 2). Asked zero-shot to write PyOpticL code for an AOM double pass or SAS setup, the model used a stale or invented API: 1/18, 1/52 and 1/42 API references were valid (95–98% hallucination). With a ∼17k-token context of real designs plus a signature digest, all references were valid (135/0, 140/0, 130/0; 0%). With the PyOpticL paper's Fig. 2a plus grounding, the model recovered 15/15 components in order, 14/15 beam indices and 14/15 rotations, but distances were 1.5–3× too short. In schema-constrained figure→JSON extraction it recovered 15/15 components with the complete beam graph and read 9/9 printed part numbers verbatim from a second figure without inventions. High-thinking calls took 73–98 s for extraction; non-streaming high-thinking calls hung beyond 600 s four times while streaming calls succeeded. These findings fixed the architecture: emit an IR instead of library code, constrain output by schema, ask for normalized boxes, correct distances deterministically, and stream.

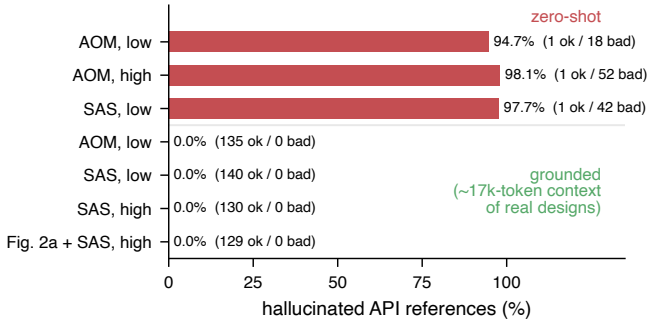


Figure 2: Grounding study: fraction of valid PyOpticL API references in generated code, zero-shot versus grounded with real designs and a signature digest (data: `experiments/argon_probe/FINDINGS.md`).

5.2 Reproduction reliability and timing

Table 4 lists measured end-to-end times. The demo SAS figure (PyOpticL Fig. 2a) passed 3/3 concurrent reproduction runs in 40.7, 55.0 and 41.5 s (scores 95–96), with vision extraction taking 18–21 s and synthesis 19–22 s; two runs needed no repair. For the optical-tweezers figure of Gieseler et al. [3] fetched by URL, a run (arXiv:2004.05246) passed at 95.4 in 73 s with 14/14 components matched (`demo/DEMO.md`), and after introducing the hard deadline, three tweezers reproduction runs passed in 55–74 s (reported in the message of commit `a008d56`; we did not find their raw streams in the repository). Not every run is fast: a recorded console run took 89.2 s (score 96), and an earlier check run took 118.4 s and passed with a score of only 80 (`demo/live_tweezers_check.sse`).

End to end. In two live end-to-end runs (`demo/E2E_TIMINGS.md`), a paper search to a finished build document—figure fetch, vision, synthesis, verify/autofix/repair, fly-in, as-built misalignment and tuning tour, cover render, and Typst build document, with no user input after one click—took 1:23–1:40 against a 3:00 budget. Run A (query "optical tweezers", Gieseler et al. Fig. 5) showed the PDF 99.9 s after the search click, timed by browser automation; run B reached DONE at 1:23 on the in-app run clock, recorded in the demo video. The range depends on where the clock starts (search click versus "Build this setup").

Figure 3 shows the per-agent timeline of a tweezers

Table 4: Measured end-to-end times (sources in parentheses: D = `demo/DEMO.md`; R = `demo/reliability/*.sse`; C = recorded `demo/*.sse`; G = git log; B = `optics/build_doc.py` docstring; S = `paper/scripts/scaling_data.json`; E = `demo/E2E_TIMINGS.md`).

Task	Time
Goal → verified design, best of 4, scores 97–98 (D)	45–85 s
Figure → design, SAS, 3/3 concurrent pass (R)	40.7–55.0 s
Figure → design, range over demo runs (D)	41–92 s
Tweezers by URL, 3/3 pass after deadline fix (G)	55–74 s
Tweezers by URL, 14/14 matched, score 95.4 (D)	73 s
Tweezers by URL, console recording, score 96 (C)	89.2 s
Tweezers check run, score 80 (C)	118.4 s
Hierarchical, 6 modules, 96 optics, score 98.5 (D)	∼100 s
Trace+verify, 121-element apparatus (D, S)	6.1–9.6 ms
Alignment tuning, AOM double pass, 75 steps (D)	35 ms
Build-document compile, Typst (B)	∼0.3 s
Cover art, faithful / cinematic (D)	∼10 / 15–30 s
Paper search → build PDF, 2 live runs (E)	1:23–1:40

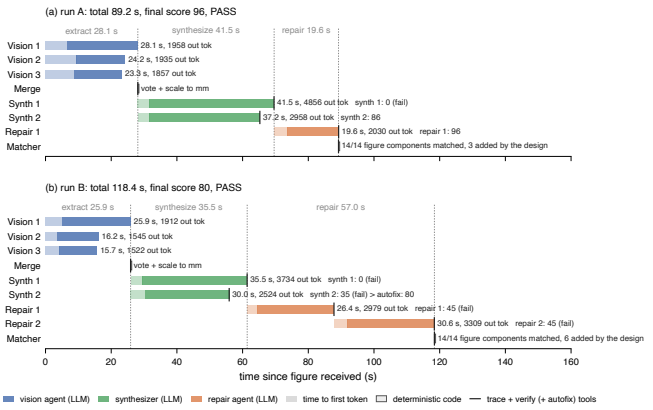


Figure 3: Per-agent timeline of two live figure→design runs on the optical-tweezers figure of Gieseler et al. [3] (arXiv:2004.05246, Fig. 5), reproduce stage only (no tune, cover or PDF tail). (a) Three parallel vision passes (23.3–28.1 s) agree 3/3 on 16 parts; the two parallel synthesizers score 0 and 86; one repair round reaches 96; total 89.2 s with 14/14 components matched. (b) Autofix lifts synthesizer 2 from 35 to 80; both repair rounds regress to 45 and are rejected, so the 80-point passing design is kept at 118.4 s. Bars are reconstructed from each agent's own elapsed time and the per-stage timings (the streams carry no wall-clock timestamps); light segments show time to first token; ticks mark deterministic trace/verify/autofix calls (each <4 ms).

run and Fig. 4 the score trajectory across candidates and repair rounds.

5.3 Eight-figure benchmark

To stress the pipelines beyond the demo we selected eight open-access setup figures of increasing difficulty plus one goal-only task (`demo/figures/RESULTS.md`, licenses in `catalog.json`); results are in Table 3 and Fig. 5. Ten early runs were cut off by a development server that restarted on file saves; they are excluded as infrastructure aborts, not counted as failures. Four of eight reproductions passed verification; three of eight were physically faithful. The fourth pass (Raman backscatter) routes the 532 nm laser line straight to the detector, because the long-pass filter was modelled as a dichroic that transmits it—a physics shortcut our verifier does not catch. The faithful reproductions include the hardest passing case, a 22-element dual-trap optical tweezer (arXiv:2411.16256; distinct from the Gieseler et al. tutorial used in the paper-search runs) with the correct split/recombine topology. All 4/4 completed goal-design runs passed.

Failure analysis. The failures cluster into six patterns, several of which led to same-day fixes: (i) *unterminated ports* on multi-laser figures (9 stray-beam errors on Sr^+) with the time budget spent before repair—addressed by the deterministic autofix and complexity-scaled budgets (commit `2707604`); (ii) *branch-topology errors* around AOMs and dumps, where one invented branch cascades into ∼15 errors—addressed by prompt rules on AOM orders and terminators; (iii) *physics shortcuts* because the IR lacks a sample/scatterer primitive—Raman, tweezer QPDs and MALDI all pass with physically wrong detection, which the verifier does not catch; (iv) *disconnected modules* in hierarchical designs, e.g. 5740 mW of module input for a 3 W laser—a power-budget double counting fixed by the deterministic budget (commit `a007e29`), after which a 3 W laser feeding 6 modules balanced exactly; (v) *numeric constraints in the notes ignored* (focal lengths, power); and (vi) *dense or 3D figures* timing out in vision—the first-pass cap was subsequently raised to 150 s, but the benchmark has not been re-run since.

5.4 Scaling

Tracing is fast enough to be invisible to agents and users (Fig. 6). Across the 13 example designs the median trace+verify time over 5 repetitions is 0.13–0.84 ms for designs of 5–28 elements; the 121-element, 7-subsystem Sr^+ apparatus (Fig. 7) takes 6–10 ms (6.1 ms on an idle machine, 9.6 ms median under concurrent load). Tiling the fixed AOM double pass up to 33 copies (363 elements, 528 segments) takes ≈105 ms median under the same load, with an empirical log–log slope of about 1.6 and an identical score (97.9) for every tiling (`paper/scripts/scaling_data.json`). Absolute

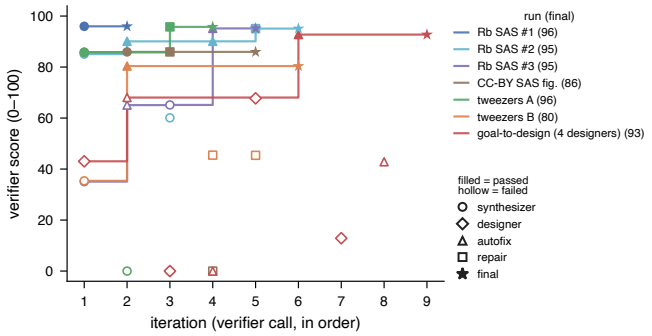


Figure 4: Verifier score for every design scored in seven live runs (best so far, i.e. the design the pipeline keeps). Marker shape gives the origin (LLM synthesizer/designer, deterministic autofix, LLM repair); hollow markers failed verification. Autofix alone produces jumps such as 35→80 and 65→95; repair adds 5–10 points (86→96, 90→95); regressing repairs are discarded.

times carry roughly ±2× load noise.

5.5 Alignment tuning

We detuned three designs with `detune(seed=1)` and ran the tuner (Fig. 8; `paper/scripts/tuning_data.json`). For the fixed AOM double pass, delivered fiber power fell from 11.80 to 3.73 mW and was recovered to 12.06 mW (102% of the reference) in 75 steps (35 ms in the demo measurement). For a fiber-coupling telescope, 1.09 mW was recovered to 9.82 mW (reference 9.64) in 63 steps. For the SAS setup the detuning barely changed delivered power (18.07 vs. 18.32 mW); the tuner still raised the objective from 2.77 to 3.18 in 67 steps while delivered power ended at 18.00 mW, because the objective also weighs score and targets. Recovery above 100% means the hand-written reference was itself slightly misaligned.

5.6 Optical logic

As a probe of the nonlinear models we built optical logic examples (`demo/OPTICAL_LOGIC.md`; Fig. 9). Linear "directed logic" gates encode bits as half-wave-plate angles (0ř or 45ř) steering a single rail through PBSs; NAND, half adder and full adder pass their truth tables. The all-optical SFG half adder takes two laser inputs: CARRY is the sum-frequency beam and SUM the residual fundamental. With inputs A = 1064 nm at 218.5 mW and B = 1550 nm at 150 mW, CARRY is 631 nm light. Its truth table (threshold 40 mW) gives SUM = 150 and 218.5 mW for single inputs and, for $A = B = 1$, CARRY = 347.4 mW with SUM depleted to 21.1 mW. An SFG AND gate with 50 mW per input gives 35.3 mW only for $A = B = 1$ (threshold 0.5 mW) (`paper/scripts/logic_truth.json`). These rely on idealized models: the SOA NAND has ∼5 dB contrast and is not cascadable in our model, and the half adder assumes a single PPLN crystal with optimistic efficiency.

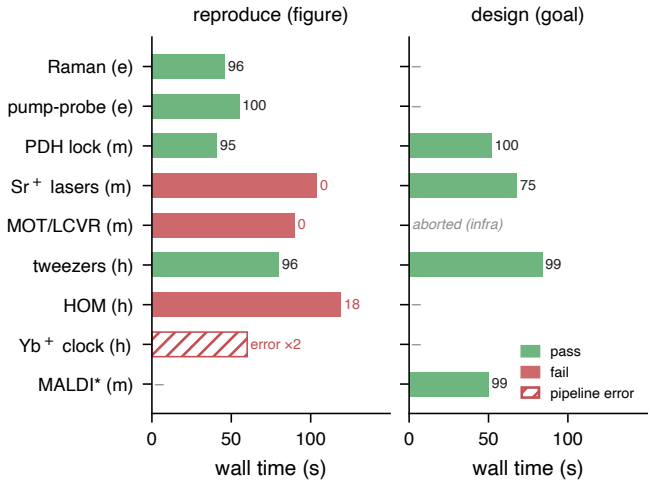


Figure 5: Stress test on eight open-access setup figures: wall time (bars) and verifier score (labels); green PASS, red FAIL. Reproduce: 4/8 pass verification (3/8 physically faithful). Goal design: 4/4 completed runs pass; one MOT design run aborted by a dev-server reload is excluded (data: `demo/figures/runs/summary.json`, `RESULTS.md`).

5.7 Build kit

For the fixed AOM double pass the build kit lists 18 purchased parts and 58 hardware items at an estimated $4.1k, and two watertight baseplate tiles requiring an estimated 38 h and 566 g of PLA to print (`demo/DEMO.md`). Build documents generated for four designs are in `demo/build_docs/`. Figure 10 shows representative outputs.

6 Discussion: the potential of multi-agent systems for optical design

Decomposition matches how laboratories modularize. Real apparatus is built as fiber-linked subsystems—a laser head, a double pass, a spectroscopy lock, a delivery board—which is exactly the decomposition that PyOpticL baseplates encourage [7] and that our architect produces. Splitting a 100-element problem into modules of 10–20 elements keeps each agent's context and output within the regime where the model is reliable, and lets modules be designed in parallel. The merge step showed that decomposition is only as good as its interface contract: without deterministic power accounting, module agents happily invented their own lasers.

Parallel exploration against an objective verifier. Because tracing and verifying costs milliseconds, it is cheap to generate several candidates and keep the best, and to run several vision or synthesizer agents concurrently. The verifier turns "is this a good layout?"—a

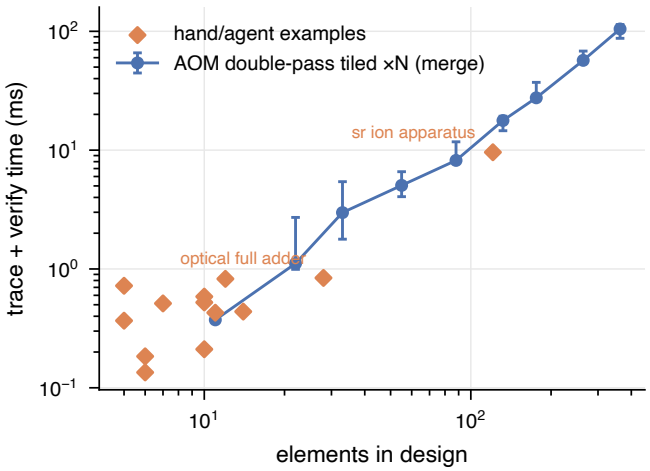


Figure 6: Trace+verify time versus number of elements: example designs and tiled copies of the AOM double pass (median of 5; `paper/scripts/scaling_data.json`).

question LLMs answer unreliably—into a score and a list of named defects that are easy for the next agent to act on. This is the key difference from LLM-critic loops [26, 27]: the critic cannot be talked into agreement.

Consensus for perception. Reading an optics schematic is a perception task with characteristic errors (missed small mirrors, misread splits). Running three differently-prompted vision agents and voting on the parts signature cost little extra latency (the passes run concurrently) and, in the recorded SAS runs, two of three agreed on a 14-part list while the third overcounted mirrors. Deterministic post-processing (normalized boxes, mm scaling from known mount sizes) corrects the systematic distance compression found in the probe.

Deterministic tools as ground truth. Our most effective fixes were not prompt changes but deterministic tools that took work away from the model: stray-beam autofix, power budgets, region packing, mm scaling and component matching. The model is left with what it is good at—recognizing structure and proposing topology—and the arithmetic and geometry are owned by code. The grounding study points the same way: emitting a small, schema-constrained IR is far more robust than emitting library code.

Human in the loop. The live console, the iteration timeline with diffs, and the editable 3D bench make every agent decision inspectable and reversible. We see the system as a drafting assistant that brings an expert to a verified first layout in about a minute, not as a replacement for the expert's judgement.

Failure modes. The benchmark shows where the approach is still weak: topology and branch-naming errors that cascade; layout drift when notes specify numbers the model replaces with "convenient" ones; and physics shortcuts that the verifier does not detect, notably missing sample or scatterer elements so that detectors observe the wrong beam. A verifier rewards only what it checks; a high score on a wrong design is the most dangerous failure, and expanding the verifier (e.g. "a detector must not receive the excitation wavelength") is as important as improving the agents.

Toward closed loop. Uddin et al. [19] demonstrate robotic assembly and alignment of AI-designed free-space optics. Beamline's keyed baseplates and alignment tuner suggest a path to a closed loop in which the tuner's knobs are driven by real power-meter or camera feedback, and discrepancies between measured and simulated power update the model of the bench.

Benchmarks are needed. Our eight-figure set is small and hand-curated. Progress requires standardized datasets of setup diagrams paired with ground-truth component graphs and layouts, with clear licenses (many figures are not openly licensed), and metrics that score physical correctness rather than verifier pass rates alone.

7 Limitations

The physics model is two-dimensional (all optics in one plane, no periscopes or vertical steering); Gaussian beams are round and lenses thin; there is no interference, coherence or cavity modelling, so interferometers and resonators are represented only geometrically; nonlinear models are steady-state, CW and textbook (perfect phase matching, no overlap integrals or acceptance bandwidths); prices are list-price estimates; and print times are estimates. The evaluation was performed on a single day with a single model family, small sample sizes (3 reliability runs per figure, 8 benchmark figures), timing measured on one laptop against a live API, and benchmark runs that predate later pipeline changes. The verifier's pass criterion does not establish physical correctness, as the Raman and tweezers cases show.

8 Conclusion

Beamline shows that a team of LLM agents, held to account by a fast deterministic tracer and verifier, can turn a goal or a published figure into a verified, tuned and fabricable tabletop optical layout in about a minute, and can scale to a 121-element apparatus through hierarchical decomposition. The same experiments show where the frontier lies: perception of dense figures, faithful use of numeric constraints, and physics that the verifier does not yet check. We believe multi-agent systems are a natural fit for optical system design precisely because optics

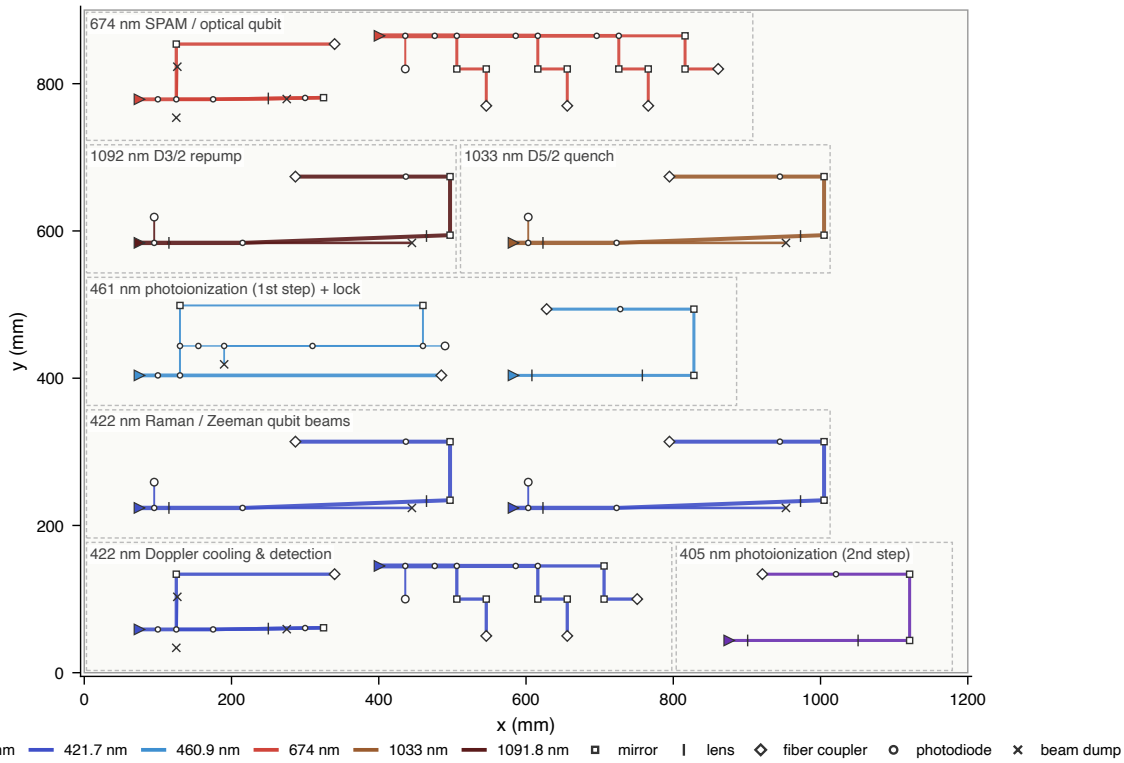


Figure 7: Plan view of the hierarchical Sr^+ ion-trap laser apparatus drawn directly from the deterministic trace: 121 elements, 11 sources, 134 beam segments, 6 wavelengths (405–1092 nm) on a 1200 × 900 mm table split into 7 module regions (dashed). Color encodes wavelength, line width $\sqrt{\text{power}}$. The design passes verification (score 99.2), delivering 236.9 of 280.0 mW input.

offers what language alone lacks—an objective, cheap, deterministic judge.

Data and code availability

All code is in the `beamline` repository: physics in `backend/optics/`, agents in `backend/agents/`, examples in `backend/examples/`, web app in `web/`, and the API and event contract in `CONTRACT.md`. Measured numbers come from `demo/DEMO.md`, `demo/figures/RESULTS.md` (raw streams in `demo/figures/runs/`), `demo/reliability/*.sse`, `demo/*.sse`, `experiments/argon_probe/FINDINGS.md` (in the parent hackathon directory), and `paper/scripts/*.json`; figure scripts are in `paper/scripts/`. End-to-end paper-to-PDF timings are recorded in `demo/E2E_TIMINGS.md`, with the demo video at `demo/video/beamline_demo_60s.mp4` (script in `demo/video/SCRIPT.md`). The project page is https://elevenarms.github.io/beamline/. Source figures and licenses are listed in `demo/figures/catalog.json`; raw arXiv downloads are not redistributed.

Acknowledgments

Agents use Google Gemini `gemini-4-argon` [11]; cover renders use Nano Banana 2 and 2.1. The software, this paper, its figures and its bibliography were produced with AI coding assistance (Claude Code and sub-agents) under the team's direction during the hackathon. We thank the PyOpticL authors [7] for the open-source library and concept that inspired this work, and the authors of the openly licensed figures used in our benchmark.

References

[1] Colin D. Bruzewicz, John Chiaverini, Robert McConnell, and Jeremy M. Sage. Trapped-ion quantum computing: Progress and challenges. *Applied Physics Reviews*, 6(2):021314, 2019. doi: 10.1063/1.5088164.

[2] A. Ashkin, J. M. Dziedzic, J. E. Bjorkholm, and Steven Chu. Observation of a single-beam gradient force optical trap for dielectric particles. *Optics Letters*, 11(5):288, 1986. doi: 10.1364/OL.11.000288.

[3] Jan Gieseler, Juan Ruben Gomez-Solano, Alessandro Magazzù, Isaac Pérez Castillo, Laura Pérez García, Marta Gironella-Torrent, Xavier Viader-Godoy, Felix Ritort, Giuseppe Pesce, Alejandro V. Arzola, Karen Volke-Sepúlveda, and Giovanni Volpe. Optical tweezers—from calibration to applications: a

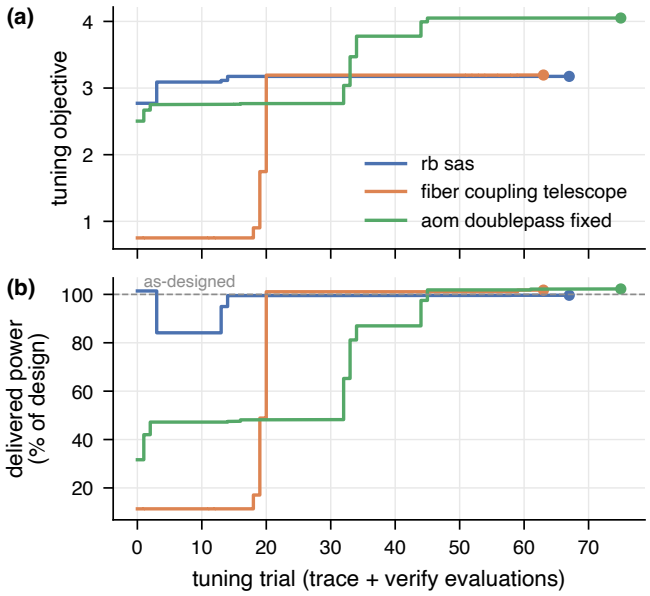


Figure 8: Deterministic re-alignment after seeded misalignment (seed 1). (a) Best-so-far objective versus trial; (b) delivered power relative to the as-designed value: fiber telescope 11%→102%, AOM double pass 32%→102%, SAS essentially unchanged (`paper/scripts/tuning_data.json`).

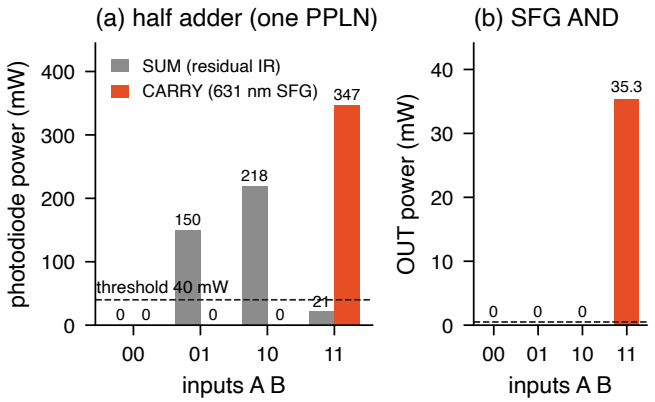


Figure 9: (a) All-optical half adder on one PPLN crystal (A = 1064 nm, 218.5 mW; B = 1550 nm, 150 mW): SUM (residual IR) and CARRY (631 nm SFG) for inputs 00/01/10/11; all rows correct at a 40 mW threshold. (b) SFG AND gate, 50 mW per input: 35.3 mW only for 11 (threshold 0.5 mW) (`paper/scripts/logic_truth.json`).

tutorial. *Advances in Optics and Photonics*, 13(1): 74, 2021. doi: 10.1364/AOP.394888.

[4] E. A. Donley, T. P. Heavner, F. Levi, M. O. Tataw, and S. R. Jefferts. Double-pass acousto-optic modulator system. *Review of Scientific Instruments*, 76 (6):063112, 2005. doi: 10.1063/1.1930095.

[5] Daryl W. Preston. Doppler-free saturated absorption: Laser spectroscopy. *American Journal of Physics*, 64(11):1432–1436, 1996. doi: 10.1119/1.18457.

[6] Eric D. Black. An introduction to Pound–Drever–Hall laser frequency stabilization. *American Journal of Physics*, 69(1):79–87, 2001. doi: 10.1119/1.1286663.

[7] Jacob Myers, Christopher Caron, Nishat Helaly, Zhenyu Wei, Justin Oh, Zack Gotobed, Kotaro Yabe, and Robert J. Niffenegger. Qubit operations using a modular optical system engineered with PyOpticL: a code-to-CAD optical layout tool. *Quantum*, 10:2135, 2026. doi: 10.22331/q-2026-06-15-2135.

[8] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023. doi: 10.1038/s41586-023-06792-0.

[9] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D. White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5): 525–535, 2024. doi: 10.1038/s42256-024-00832-8.

[10] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292, 2024.

[11] Gemini Team. Gemini: A family of highly capable multimodal models. arXiv preprint arXiv:2312.11805, 2023.

[12] A. Freise, G. Heinzel, H. Lück, R. Schilling, B. Willke, and K. Danzmann. Frequency-domain interferometer simulation with higher-order spatial modes. *Classical and Quantum Gravity*, 21(5): S1067–S1074, 2004. doi: 10.1088/0264-9381/21/5/102. Current software: Finesse 3, https://finesse.ifosim.org/, doi:10.5281/zenodo.821363.

[13] Alexander Franzen. ComponentLibrary: a vector graphics library for illustrations of optics experiments. https://www.gwoptics.org/ComponentLibrary/, 2026. Licensed CC BY-NC 3.0. Accessed 2026-10-25.

[14] Mario Krenn, Mehul Malik, Robert Fickler, Radek Lapkiewicz, and Anton Zeilinger. Automated search for new quantum experiments. *Physical Review Letters*, 116(9):090405, 2016. doi: 10.1103/PhysRevLett.116.090405.

[15] Alexey A. Melnikov, Hendrik Poulsen Nautrup, Mario Krenn, Vedran Dunjko, Markus Tiersch, Anton Zeilinger, and Hans J. Briegel. Active learning machine learns to create new quantum experiments. *Proceedings of the National Academy of Sciences*, 115(6):1221–1226, 2018. doi: 10.1073/pnas.1714936115.

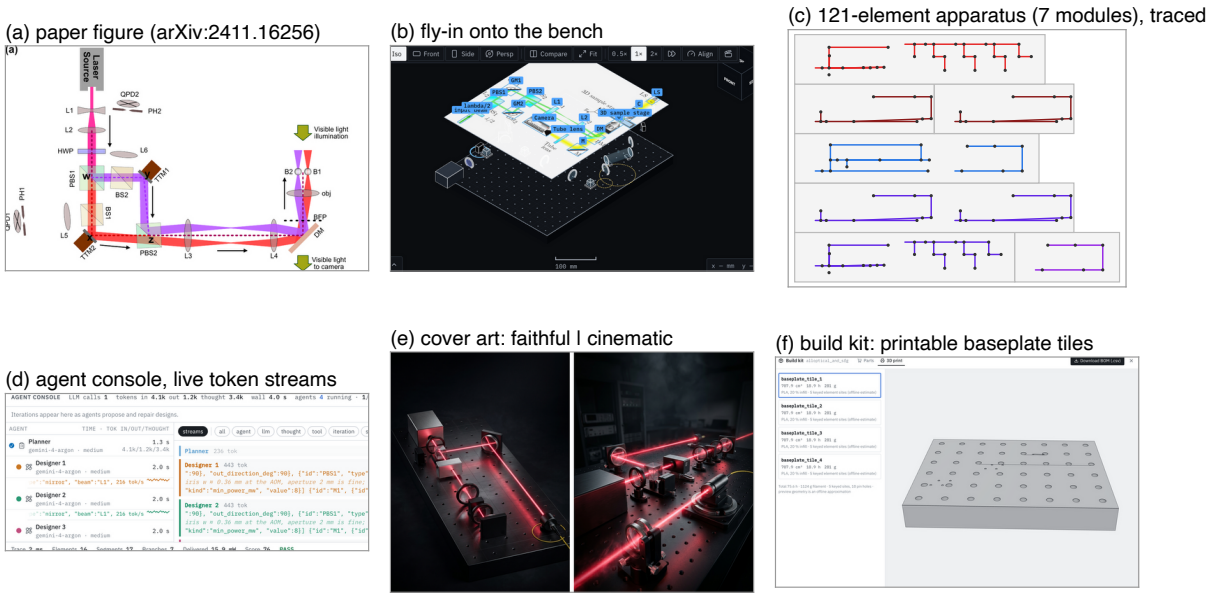


Figure 10: Artifacts produced by Beamline. (a) An open-access setup figure (dual-trap tweezers, arXiv:2411.16256, CC BY 4.0). (b) Fly-in: the figure floats above the bench as its parts are placed. (c) The 121-element Sr^+ apparatus, a plan view drawn from the trace (not an app screenshot). (d) The agent console streaming tokens from parallel agents. (e) Cover renders, faithful and cinematic (Nano Banana models). (f) Printable keyed baseplate tiles from the build kit.

[16] Carlos Ruiz-Gonzalez, Sören Arlt, Jan Petermann, Sharareh Sayyad, Tareq Jaouni, Ebrahim Karimi, Nora Tischler, Xuemei Gu, and Mario Krenn. Digital discovery of 100 diverse quantum experiments with PyTheus. *Quantum*, 7:1204, 2023. doi: 10.22331/q-2023-12-12-1204.

[17] Mario Krenn, Robert Pollice, Si Yue Guo, Matteo Aldeghi, Alba Cervera-Lierta, Pascal Friederich, Gabriel dos Passos Gomes, Florian Häse, Adrian Jinich, AkshatKumar Nigam, et al. On scientific understanding with artificial intelligence. *Nature Reviews Physics*, 4(12):761–769, 2022. doi: 10.1038/s42254-022-00518-3.

[18] Sören Arlt, Xuemei Gu, and Mario Krenn. Towards autonomous quantum physics research using LLM agents with access to intelligent tools. arXiv preprint arXiv:2511.11752, 2025.

[19] Shiekh Zia Uddin, Sachin Vaidya, Shrish Choudhary, Zhuo Chen, Raafat K. Salib, Luke Huang, Dirk R. Englund, and Marin Soljačić. AI-driven robotics for optics. arXiv preprint arXiv:2505.17985, 2025.

[20] S. K. Rithvik. Anubuddhi: A multi-agent AI system for designing and simulating quantum optics experiments. arXiv preprint arXiv:2512.15736, 2025.

[21] Yuyu Geng, Lei Sun, Yao Gao, Xinxin Hu, Zhonghua Yi, Xiaolong Qian, Weijian Hu, Jian Bai, and Kaiwei Wang. OPTIAGENT: A physics-driven agentic framework for automated optical design. arXiv preprint arXiv:2602.23761, 2026.

[22] Sean Molesky, Zin Lin, Alexander Y. Piggott, Weiliang Jin, Jelena Vučković, and Alejandro W. Rodriguez. Inverse design in nanophotonics. *Nature Photonics*, 12(11):659–670, 2018. doi: 10.1038/s41566-018-0246-9.

[23] Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, et al. Scientific discovery in the age of artificial intelligence. *Nature*, 620(7972):47–60, 2023. doi: 10.1038/s41586-023-06221-2.

[24] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.

[25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[26] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[27] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri

Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[28] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155, 2023.

[29] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

[30] H. Kogelnik and T. Li. Laser beams and resonators. *Applied Optics*, 5(10):1550, 1966. doi: 10.1364/AO.5.001550.

[31] Anthony E. Siegman. *Lasers*. University Science Books, Mill Valley, CA, 1986. ISBN 978-0-935702-11-8.

[32] Sidney A. Self. Focusing of spherical Gaussian beams. *Applied Optics*, 22(5):658, 1983. doi: 10.1364/AO.22.000658.

[33] R. Clark Jones. A new calculus for the treatment of optical systems. I. Description and discussion of the calculus. *Journal of the Optical Society of America*, 31(7):488, 1941. doi: 10.1364/JOSA.31.000488.

[34] K. E. Stubkjaer. Semiconductor optical amplifier-based all-optical gates for high-speed optical processing. *IEEE Journal of Selected Topics in Quantum Electronics*, 6(6):1428–1435, 2000. doi: 10.1109/2944.902198.

[35] M. M. Fejer, G. A. Magel, D. H. Jundt, and R. L. Byer. Quasi-phase-matched second harmonic generation: tuning and tolerances. *IEEE Journal of Quantum Electronics*, 28(11):2631–2654, 1992. doi: 10.1109/3.161322.

[36] Jian Wang, Junqiang Sun, and Qizhen Sun. Single-PPLN-based simultaneous half-adder, half-subtracter, and OR logic gate: proposal and simulation. *Optics Express*, 15(4):1690, 2007. doi: 10.1364/OE.15.001690.

[37] Emmett Lalish and Manifold contributors. Manifold: Geometry library for topological robustness (python package `manifold3d`). https://github.com/elalish/manifold, 2026. Accessed 2026-10-25.

[38] Typst GmbH. Typst: A markup-based typesetting system. https://typst.app/, 2026. Source: https://github.com/typst/typst. Accessed 2026-10-25.

[39] three.js authors. three.js: JavaScript 3D library. https://threejs.org/, 2026. Source: https://github.com/mrdoob/three.js. Accessed 2026-10-25.